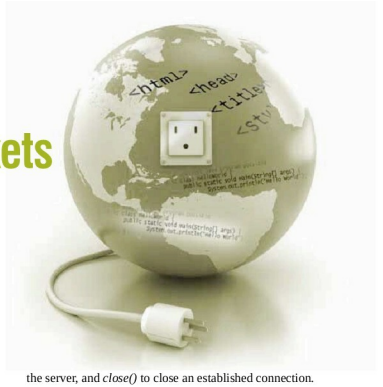
Implementing HTML5 WebSockets Using Java



Intended for those who know the basics of HTML, JavaScript, Java and JSP, this article shows readers how to use the new HTML5 WebSocket API, which allows full duplex communication between client and server.

If you open a connection to a server with the new *ws* protocol (unlike AJAX, where clients poll the server for updates), WebSocket allows the server to send messages to the clients—to *push* data instead of *pulling* it.

Polling, long polling, streaming, and WebSocket

In polling, the client polls the server at regular intervals asking for updates; the server replies with any updated content if available. In long polling, the client makes a connection to the server, which is kept alive until the server has an update. After the update is sent to the client, the connection is re-established and held open until another update is obtained from the server.

In streaming, the client creates a connection to the server and this connection is never ended; updates are sent to the clients indefinitely. Now with WebSockets, the client makes a new connection with the *ws* protocol; the server registers the client and then pushes data to it when an update is available.

Client-side API

The WebSocket object attribute *websocketobjext.readystate* shows the different stages of connection establishment; the possible values and meanings are: 1) The connection is not yet established; 2) The connection is established and communication is possible; 3) The connection is at the stage of closing the handshake; and 4) The connection is closed. The *websocketobject.bufferedamount* attribute shows the number of bytes queued to send.

WebSocket events include *onopen()*, triggered when a connection is opened; *onclose()*, when it is closed; *onerror()*, when any error occurs; and *onmessage()*, when a new message is received from the server.

WebSocket methods include *send()* to send a message to the server, and *close()* to close an established connection.

Server-side implementation

You need a server-side implementation for WebSocket to work, of course, and for this you use *WebSocketServlet*, a new type of servlet available in Tomcat 7.0.32; see the API documentation at *http://bit.ly/W6sR16*.

WebSocketServlet methods include *verifyOrigin(String origin)*, which helps check the origin of the message (the domain name of the server to which the client is connected) letting you allow or deny messages from a list of servers. Then there's *createWebSocketInbound(String subProtocol, HttpServletRequest request)*, invoked when a client makes a new WebSocket connection request; it returns an object of type *streaminbound*, representing a connection. The API doc is at http://bit.ly/V7Lt66.

Its methods include *onopen(wsoutbound outbound)*, invoked when a new connection is established; *onclose(int status)*, if an existing connection is closed; *ontextmessage(charbuffer message)*, when a client sends a text message to the server; *onbinarymessage(bytebuffer message)*, likewise, but message is binary.

The requirements to be able to use this are Tomcat 7.0.32, JDK (a recent version), and Firefox/Chrome. An example of a simple chat application is available at *http://www.linuxforu.com/article_source_code/feb13/* END

By: Shyam Chandran

The author is a Web developer based in Kochi. He is an open source enthusiast, bordering on being an evangelist. He enjoys drinking lots of coffee, hacking through the night on his Tux machine, and going on nice long rides on his Royal Enfield. Queries? Please contact him at *shyamchandranmec@gmail.com*.